

CS 201: Data Structures & Algorithms

Spring Semester 2026 · 3 Credit Hours

Instructor: Dr. Patricia Nguyen	Email: p.nguyen@university.edu	Office: Thornton Hall 214
Lecture: Mon/Wed/Fri 10:00–10:50 AM	Location: Engineering Bldg 101	Office Hours: Tue/Thu 2:00–4:00 PM

COURSE DESCRIPTION

This course provides an in-depth study of fundamental data structures and the algorithms that operate on them. Students will examine the design, analysis, and implementation of structures including arrays, linked lists, stacks, queues, trees, heaps, hash tables, and graphs. Emphasis is placed on algorithmic efficiency, Big-O analysis, and selecting appropriate data structures for real-world problem-solving. Programming assignments are completed in Java or Python.

Prerequisites: CS 101 (Introduction to Programming) and MATH 151 (Discrete Mathematics)

GRADING & ASSESSMENT

Component	Weight	Details
Programming Assignments (6)	30%	Weekly coding projects submitted via GitHub Classroom
Midterm Exam	20%	Closed-book; covers Weeks 1–5 (Week 6, Friday)
Final Exam	25%	Comprehensive; scheduled during finals week
Quizzes (8)	15%	Short in-class quizzes, lowest score dropped
Participation & Attendance	10%	In-class exercises and engagement

WEEKLY COURSE SCHEDULE

Week 1: Course Introduction & Review of Complexity Analysis

- Course overview and learning objectives
- Review of recursion and iterative thinking
- Introduction to Big-O, Big-Θ, and Big-Ω notation
- Time vs. space complexity trade-offs

■ Assignment 1 Released: Complexity Analysis Problems

Week 2: Arrays, Dynamic Arrays & Linked Lists

- Static vs. dynamic arrays; amortized analysis
- Singly and doubly linked lists
- Sentinel nodes and circular linked lists
- Array vs. linked list performance comparison

W0
3

Week 3: Stacks & Queues

- Stack ADT: array-based and linked list implementations
- Applications: expression parsing, backtracking
- Queue ADT: circular array and linked list implementations
- Deque and priority queue introduction

■ [Assignment 1 Due](#) | [Assignment 2 Released: Stacks & Queues](#)

W0
4

Week 4: Recursion & Divide-and-Conquer

- Advanced recursion: memoization and tail recursion
- Merge sort and quicksort deep dive
- Recurrence relations and the Master Theorem
- Practical sorting benchmarks

W0
5

Week 5: Trees: Binary Trees & Binary Search Trees

- Tree terminology: height, depth, balance, traversals
- BST operations: insert, search, delete
- In-order, pre-order, post-order traversals
- BST performance: average vs. worst case

■ [Assignment 2 Due](#) | [Assignment 3 Released: BSTs](#)

W0
6

Week 6: Balanced Trees: AVL & Red-Black Trees

- Motivation for self-balancing trees
- AVL trees: rotations and rebalancing
- Red-black tree properties and operations
- Comparison of balanced tree implementations

■ **MIDTERM EXAM — Friday of Week 6 (Covers Weeks 1–5)**

W0
7

Week 7: Heaps & Priority Queues

- Binary heap structure and heap property
- Heapify, insert, and extract-min/max operations
- Heap sort algorithm
- Applications: Dijkstra's algorithm, task scheduling

■ [Assignment 3 Due](#) | [Assignment 4 Released: Heaps](#)

W0
8

Week 8: Hash Tables & Hashing

- Hash functions: design principles and distribution
- Collision resolution: chaining vs. open addressing
- Load factor, rehashing, and table resizing
- Applications and practical hash table usage

W0
9

Week 9: Graphs: Representation & Traversal

- Graph terminology: directed, undirected, weighted, DAGs
- Adjacency matrix vs. adjacency list representations
- Breadth-first search (BFS) and applications
- Depth-first search (DFS) and topological sort

■ Assignment 4 Due | Assignment 5 Released: Graph Traversal

W1
0

Week 10: Graph Algorithms & Course Wrap-Up

- Shortest path algorithms: Dijkstra's and Bellman-Ford
- Minimum spanning trees: Kruskal's and Prim's
- Introduction to dynamic programming
- Course review, algorithm design patterns, next steps

■ Assignment 5 Due | Assignment 6 (Final Project) Released

COURSE POLICIES

Late Work

Assignments submitted late will be penalized 10% per day, up to a maximum of 50%. No submissions accepted after 5 days past the deadline. Exceptions require documented emergencies communicated before the due date.

Academic Integrity

All submitted work must be your own. Collaboration on assignments is permitted for discussion purposes only — code must be written independently. Use of AI code generation tools must be disclosed. Violations will be reported to the Academic Integrity Committee.

Attendance

Regular attendance is expected. More than four unexcused absences may result in a grade reduction. Students are responsible for all material covered in missed sessions.

Accessibility

Students requiring accommodations should contact the Disability Services Office and provide documentation to the instructor within the first two weeks of class.

REQUIRED MATERIALS

Textbook (Required): Cormen, Leiserson, Rivest & Stein — *Introduction to Algorithms*, 4th Edition (MIT Press, 2022)

Supplemental: Sedgewick & Wayne — *Algorithms*, 4th Edition (Addison-Wesley). Free online resources and lecture slides will be posted to the course portal.

Software: Java 17+ (OpenJDK) or Python 3.10+, IntelliJ IDEA or VS Code, Git